IFIP AICT 576



Marco A. Wehrmeister
Márcio Kreutz
Marcelo Götz
Stefan Henkler
Andy D. Pimentel
Achim Rettberg (Eds.)

Analysis, Estimations, and Applications of Embedded Systems

6th IFIP TC 10 International
Embedded Systems Symposium, IESS 2019
Friedrichshafen, Germany, September 9–11, 2019
Revised Selected Papers

Springer



Workload Characterization for Memory Management in Emerging Embedded Platforms

Biswadip Maity(✉), Bryan Donyanavard, Nalini Venkatasubramanian, and Nikil Dutt

University of California, Irvine, USA
{maityb,bdonyana,nalini,dutt}@uci.edu

Abstract. Memory has emerged as a primary performance and energy bottleneck for emerging embedded platforms that integrate heterogeneous compute units. Applications require a balance between performance and energy-efficiency and finding the optimal operating point on embedded platforms is challenging. There exist many opportunities to manage the memory subsystem efficiently at runtime to save energy without compromising quality in the face of dynamic workloads. Previous works have used memory bandwidth utilization to determine memory requirements and develop runtime policies to configure system knobs (e.g., memory controller frequency) accordingly. However, bandwidth utilization as a singular metric is not always sufficient: policies for a range of workload scenarios require insight into an application's memory access pattern and working set size. Alternatively, memory profilers provide fine-grained information such as the memory access pattern for the entire virtual address space, and the load/store density of different regions of the memory. However, parsing this detailed information frequently at runtime induces excessive overhead. In this work, we propose a profiling mechanism that considers both (1) the working set size of running workloads and (2) memory bandwidth utilization to compute WBP (Working Set Size-Bandwidth Product). WBP can be estimated with low overhead, and the combined metric provides insights that runtime policies can use to determine desirable configurations for specific workload scenarios. Our early results show that a static configuration devised with this metric yields an optimal memory controller frequency for 8 out of 10 PARSEC workloads, demonstrating the promise of this approach.

Keywords: Main memory · Memory bandwidth · Runtime memory management

1 Introduction

Contemporary embedded systems are equipped with heterogeneous compute units with shared main memory to meet the varying memory capacity and bandwidth requirements of modern applications (e.g., Nvidia Tegra, Nvidia Xavier,

© IFIP International Federation for Information Processing 2023
Published by Springer Nature Switzerland AG 2023
M. A. Wehrmeister et al. (Eds.): IESS 2019, IFIP AICT 576, pp. 65–76, 2023.
https://doi.org/10.1007/978-3-031-26500-6_6

AMD Accelerated Processing Unit). All computational resources (e.g., CPU, GPU, DSP, on-chip accelerator) share the main memory, resulting in memory contributing as a primary performance and energy bottleneck for emerging embedded system platforms. To address this bottleneck, application developers use memory profilers to understand the runtime requirements of applications, and attempt to reduce their memory footprint through code optimization.

At runtime, resource managers monitor system state and implement policies to update system configuration (i.e., knobs) to meet the system's goals (e.g., maximize performance-per-unit-power). Traditionally, policies that target memory exploit memory bandwidth under-utilization at runtime by dynamically scaling the memory controller frequency in order to reduce power without compromising quality. However, embedded systems commonly support many hardware and software knobs which: (1) schedule applications across compute units, (2) set the frequency of heterogeneous compute units, and (3) set the degree of parallelism by changing the number of threads; all of which can affect the time required to process application data and, in turn, energy consumption.

Although memory profilers report the number of memory accesses (load/stores) and generate heatmaps (i.e., memory access density) for the profiled application, it is difficult for policies to manage the overhead of parsing this information at runtime. The overhead of analyzing the information from memory profilers makes the information impractical to consider for high-frequency runtime decision making. Thus, it becomes essential to monitor a coarse-grained metric(s), which can assist the runtime resource managers in efficiently determining the ideal system configuration.

We propose a memory-driven application classification based on multiple dimensions in order to assist in developing policies for resource allocation strategies, specifically for DRAM. We combine (a) the size of memory required (in MBs) of the working set of an application, and (b) the runtime DRAM bandwidth requirement (last-level cache miss rate) to classify an application's memory requirement. Both measurements can be made with minimal overhead and can be used at runtime to evaluate the dynamic requirements of applications. In this work, this classification is used to determine static memory controller frequency to minimize the Energy-Delay Product (EDP). However, this classification can be used generally to develop policies which decide the system configuration based on the classes of the application currently running on the system.

2 Motivation

In 1965 Gordon E. Moore predicted that manufacturers would keep doubling the number of transistors on a chip almost every two years [11]. The developments of the semiconductor industry by leaps and bounds have introduced billions of transistors on tiny dies. The law has also set the pace for software running on the underlying hardware. Small computers present in most households devices (e.g., smartwatches, smart-lights, mobile devices) and personal computers are now equipped with more resources than can be powered. With the growth in embedded compute resources, applications have now become extremely demanding.

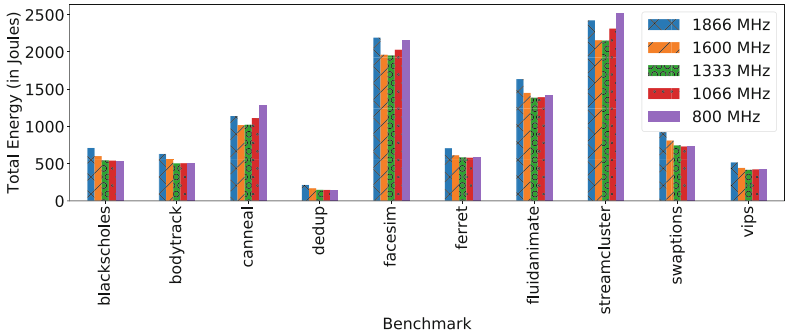


Fig. 1. Total energy consumed as memory controller frequency is varied statically from 800 MHz to 1866 MHz for PARSEC workloads on the Jetson TX2. The frequency that results in the minimum energy spent varies.

Applications generate data at a rate that tests the extremes of main memory latency and bandwidth. Thus, the performance and energy bottleneck in today's embedded system platforms is no longer the computation of data, but the transportation of that data to and from computation. One current challenge for computing systems is that the computation is performed far away from the data [7,9].

Although caching is traditionally applied to address this challenge, the working set size of applications may be significant enough to exceed the cache capacity. The growth in working set size leads to cache misses and requires main memory accesses to fetch data for the processor. The movement of data from the DRAM to the processor consumes 19% of total system power [3] and cache misses consume more than 50% of the cycles [7]. Previous works have used memory bandwidth utilization to determine memory requirements and develop runtime policies to address the issue of memory-related energy consumption. [3] proposes to set the memory controller frequency at the lowest frequency (i.e., 800 MHz) when the memory is lightly loaded (memory bandwidth is under 2000 MB/s).

Figure 1 shows the effect of memory controller frequency on total energy consumption for a variety of benchmarks executing on a contemporary embedded platform, the Nvidia Jetson TX2 [1]. A single instance of 10 workloads from the PARSEC benchmark suite [2] is executed on the Jetson TX2 with fixed memory controller frequency at various levels, and a runtime system calculates the total energy spent. The first observation we make about the workloads executed in Fig. 1 is the memory bandwidth utilization is low: it never exceeds 430 MB/s. According to the previously described policy, one would expect the lowest frequency to be the best configuration for the system for this memory bandwidth utilization.

Second, we observe that the lowest frequency leads to the minimum energy consumption for only 2 out of 10 workloads. This is due to the fact that, although the power consumption is minimal, the extended execution time results

in increased energy consumption in 8 out of 10 cases. Even from this simple example, we can conclude that in order to account for additional energy consumption due to excess latency, we must consider more than just bandwidth when a policy decides on the memory controller frequency.

3 Diversity of Working Set Size in Embedded Applications

Denning defines the working set of an application as a collection of recently referenced pages of the application's virtual address space [5]. Working set is an efficient tool to measure the memory demands of an application. A similar metric, the resident set, is defined as the subset of an application's segments which are present in main memory at a given time.

3.1 PARSEC Benchmark Suite

The PARSEC benchmark suite [2] is a popular suite for evaluating multiprocessor-based embedded platforms. It consists of twelve workloads (nine applications and three kernels) from the recognition, mining, and synthesis (RMS) domain, as well as representative embedded system applications. The workloads compose a diverse representative of working set size, degree of parallelism, off-chip traffic, and data-sharing that can be representative of the workloads executed on emerging embedded platforms. Based on the working set, two broad classes of workloads are distinguished:

1. **Working set smaller than 16 MB**: These applications do not generate much memory pressure because the working set can fit in the last-level shared cache. Example applications are `bodytrack` and `swaptions`.
2. **Working set larger than 16 MB**: These applications have a very large working set generating more off-chip memory accesses to the DRAM. Example applications are `canneal`, `ferret`, `facesim`, `fluidanimate`. When the input size grows, the working set can even reach gigabytes due to algorithms that operate on large amounts of collected data.

Note that as the number of cores increases with the degree of parallelism, so does the bandwidth requirement. `bodytrack` makes off-chip memory accesses in short, but bandwidth-intensive bursts. When several instances of the same application execute concurrently, these short bursts limit the scalability.

The sampling of benchmarks from PARSEC demonstrates highly variable memory requirements that depend on multiple factors. The number of load/store operations, size of the last-level cache, and the number of parallel threads are some of the factors that affect the bandwidth requirement. These various sources of memory pressure provide opportunities for resource managers to address the bottleneck at runtime for unpredictable workloads. Working set can effectively represent some of these dynamics, and we show that combining the working set size information with memory bandwidth at runtime can lead to efficient system configurations.

4 Related Work

Current runtime memory management metrics and policies are discussed in this section.

4.1 Runtime Policies Using Memory Bandwidth Utilization

Several techniques have been employed to address the memory bottleneck, and *Dynamic Voltage and Frequency Scaling (DVFS)* of the memory controller is a primary one [4]. Higher frequency yields higher throughput but consumes more power. DVFS can reduce the memory power by 10.4% on average, 20.5% maximum for the SPEC 2006 benchmarks without compromising quality [3]. [8] uses an approximation equation (called MAR-CSE) based on the correlation of the memory access rate and the critical frequency for the minimum energy consumption. The memory access rate is the ratio of the total number of cache misses (including both instruction and data cache misses) to the number of instructions executed. MAR-CSE predicts the voltage and frequency at runtime. Although this technique does not measure the memory bandwidth directly, the memory access rate is also an indirect measure of the memory bandwidth.

4.2 Runtime Policies Using Workload Information

Recent work [10] looks into the combined effect of application compute/memory-intensity, thread synchronization contention, and nonuniform memory accesses pattern to develop a runtime energy management technique by performing DVFS on CPU cores. However, they do not consider memory bandwidth utilization and do not change the memory controller frequency.

4.3 Runtime Policies Using Reflection or Prediction

CPU DVFS has been explored extensively in literature [13] and most systems use it to meet system goals. Linux provides several governors to manage the CPU frequency at runtime (e.g., `ondemand`, `performance`, `powersave`). Recent work uses a reflective system model to predict the system behavior when different frequencies are selected [6,12].

4.4 Runtime Policies Using Task Mapping

The operating system scheduler is responsible for (1) deciding the schedule of running applications and (2) deciding which threads run on which cores. Modern systems provide heterogeneous compute resources which have different power/performance tradeoffs. The smaller cores are more power efficient but have less computational capacity than the bigger cores. Task mapping has shown energy savings of up to 51% when combined with DVFS [14].

5 Classification Methodology

The proposed approach combines the working set size and memory bandwidth by calculating their product (WBP). The objective is to find the operating frequency, which leads to the lowest energy-delay product (EDP). Based on the EDP at different values of WBP, applications are categorized into one of three classes. In this section, the WBP metric and the classification ranges are described.

The correlation between WBP and EDP is observed experimentally. Ten PARSEC benchmarks are executed with different fixed memory controller frequencies. At regular intervals (*sensing windows*) of 200 ms, the following metrics are recorded:

1. **Working Set Size**: Linux kernel 4.3 introduced idle page flags to track memory utilization. Once a process starts, the idle bits corresponding to all the virtual pages in that process are set to 1 to indicate that they have not been referenced. Whenever the process issues memory read/write requests, the idle bit corresponding to the virtual page is set to 0 by Linux kernel. A 0 implies that the page is not idle. We selected the window size as 200 ms, however exploring more window durations and feasibility of adaptive windows remain as future work. Every 200 ms, the number of 0 bits in the idle page flags are read and used to calculate the working set size as 0 as follows:

$$WorkingSetSize = \text{Number of active pages } \times \text{ Size of each page}$$

2. **Memory Bandwidth**: ARM cores include logic to gather various statistics on the operation of the processor and memory system during runtime based on a Performance Monitoring Unit (PMU). The PMU provides hardware counters for different events, which are used to profile application behavior. The counter values of `L2_CACHE_REFILL` and `MEM_ACCESS` on each core are monitored to understand the memory traffic at runtime every 200 ms (Sensing Window Length = 0.2 s). Since the L2 is the last-level shared cache on the Jetson, the memory bandwidth can be calculated using these values as:

$$MemoryBandwidth = \frac{\sum_{i=1}^{activeCores} \texttt{L2_CACHE_REFILL}_{Core_i} \times DataBusSize}{SensingWindowLength}$$

3. **Memory Power and System Power**: Nvidia drivers read the power measurements from onboard sensors connected by `I2C`. The separate domains for system power and memory power help isolate the energy consumed by the memory separate from the rest of the system.
4. **Latency/Delay**: The amount of time that the workload takes to complete is indicative of the compute and memory latency. The CPU governor is set to 'userspace' and the frequency of all CPU cores is set to maximum for all the experiments. Thus, changes in memory latency due to the change in memory controller frequency is reflected in the total execution time.

5.1 Classification of PARSEC Benchmarks

Our objective is to find the operating frequency that results in the lowest energy-delay product. To that effect, we are interested in finding changes in EDP across frequencies at different values of WBP. This is done by experimentally observing the average EDP (over 10 applications) of PARSEC workloads on a target embedded system described in Sect. 6.1.

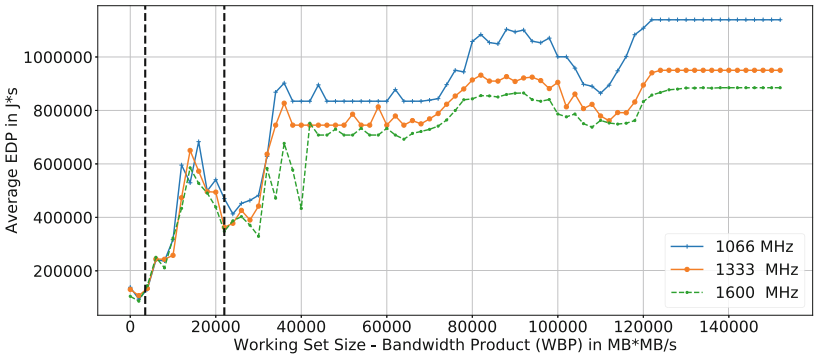


Fig. 2. Change in average EDP (energy-delay product) of PARSEC workloads across frequencies at different values of WBP (working set size - bandwidth product). (Color figure online)

The average EDP at different values of WBP for PARSEC workloads obtained experimentally are presented in Fig. 2. The different colors represent different frequencies. The PARSEC applications operate in different regions of WBP throughout execution. At low values of WBP, the EDP does not change with frequency. This is because the working set size of the application is small, and the memory requests are served from cache. Thus, operating at lower frequency does not have any effect on the EDP. As the WBP increases, memory requirement increases, and higher frequencies perform better. Frequencies lower than 1066 MHz (e.g., 800 MHz) have a very high latency whereas frequencies higher than 1333 MHz (e.g., 1866 MHz) consume too much power during the execution. Thus, they never obtain optimal EDP for any of the workloads. Hence, frequencies 800 MHz and 1866 MHz are not included in Fig. 2.

We classify applications based on their WBP profile. For each class, the goal is to select a memory controller frequency that leads to the minimum EDP. Two thresholds for WBP are selected based on the average application EDP profile at different frequencies. In this work, the operating frequency is determined statically based on the average WBP profile of workloads. However, we acknowledge the possibility of a runtime policy which checks the WBP at regular intervals to change the frequency during application execution. The classification and thresholds are proposed as follows:

1. C_1 (Small memory footprint): $0\,MB^2/s \leq WBP < 3500\,MB^2/s$ These applications have a small working set size and a low memory bandwidth. The L2

Table 1. Classification of PARSEC workloads based on average WBP (working set size - memory bandwidth product) during runtime.

Workload	WBP (in MB^2/s)	Class	EMC frequency
`dedup`	2100	C_1	1066 MHz
`swaptions`	3000	C_1	1066 MHz
`bodytrack`	3600	C_2	1333 MHz
`ferret`	3700	C_2	1333 MHz
`blackscholes`	5100	C_2	1333 MHz
`vips`	9700	C_2	1333 MHz
`fluidanimate`	16000	C_2	1333 MHz
`canneal`	22000	C_3	1600 MHz
`facesim`	62222	C_3	1600 MHz
`streamcluster`	119000	C_3	1600 MHz

cache is large enough to accommodate most of the requests to memory for this class of application. When running C_1 applications, **the system is configured at 1066 MHz**.

2. C_2 (Medium memory footprint): $3500\,MB^2/s \leq WBP < 22000\,MB^2/s$ These applications have a moderate memory requirement. The L2 cache cannot accommodate all the requests to the memory. The working set size is also considerable and generates requests which need to go to main memory. When running C_2 applications, **the system is configured at 1333 MHz**.
3. C_3 (Large memory footprint): $WBP > 22000\,MB^2/s$ These applications have a high memory requirement. Operating the system at 1600 MHz clearly gives the lowest EDP. The working set size for this class of application is large and spread out in different regions of memory. This incurs excessive cache misses and generates requests to the main memory. When running C_3 applications, **the system is configured at 1600 MHz**.

6 Evaluation

6.1 Experimental Setup

We use Jetson TX2 [1] from Nvidia, an embedded System-On-Chip (SOC) platform, to evaluate our proposed technique. The Jetson has heterogeneous compute cores (quad-core ARM Cortex A57 and dual-core Nvidia Denver2) distributed in two clusters along with an onboard 256-core Pascal GPU. Each of the clusters resides in a separate frequency domain. Jetson has a shared memory architecture, and all resources (CPU clusters, GPU) share the main memory. Jetson TX2 has an 8 GB 128-bit LPDDR4 memory and a 32 GB eMMC 5.1 for onboard storage. The Cortex cores come with: 48 KB L1 instruction cache (I-cache) per core; 32 KB L1 data cache (D-cache) per core. The Denver cores have 128 KB L1

I-cache per core; 64 KB L1 D-cache per core. All the cores share an L2 Unified Cache of 2 MB.

Jetson uses an External Memory Controller (EMC) to manage the off-chip memory traffic. The EMC has different operating frequencies ranging from `4800 KHz` to `1866 MHz`. The onboard ARM Cortex Real-time (R5) Boot and Power Management Processor (BPMP) changes the memory controller frequency through kernel drivers. During all the experiments, the Denver cores are switched off, and the Cortex A57 cores are configured at the highest frequency. Disabling CPU DVFA helps isolate the effect of EMC operating frequency.

6.2 Experimental Results

In Sect. 5, PARSEC benchmarks are classified into three classes, and an ideal memory controller frequency for each class is proposed. applications are executed with a fixed memory controller frequency based on class. The results are presented in Fig. 3. We expect to see lower EDP with the proposed approach when compared to techniques which do not utilize any memory information when deciding the EMC Frequency.

The average WBPs for PARSEC benchmarks are presented in Table 1. The working set size and memory bandwidth are measured using the Linux idle page tracker and `L2_CACHE_REFILL` PMU event counter every 200ms. The proposed static configuration uses the EDP values obtained with this method. The EDP value is compared with other static configurations (1600 MHz, 1333 MHz and 1066 MHz) which serve as baselines in Fig. 3. From the figure, we see that the proposed scheme can find the optimal configuration for eight out of ten PARSEC workloads. The proposed scheme can achieve on average 16.7% (max: 37.3%) reduction in EDP when compared to the most aggressive memory controller frequency (1866 MHz). The results are compared with an optimal scheme obtained by executing all workloads at all possible EMC frequencies and choosing the frequency that yields minimum EDP. The optimal scheme can achieve an average reduction of 17.1% (max: 39.3%) EDP when compared to the most aggressive memory controller frequency (1866 MHz).

The static method of setting the memory controller frequency is not capable of setting the optimal knob for all the applications. Benchmarks like `blackscholes` exhibit dynamic memory accesses patterns, as shown in Fig. 4. A static configuration is not sufficient to address these bursts of memory accesses. Therefore, it is necessary to perform DVFS adaptively during application execution. Additionally, if we consider a platform running multiple applications concurrently, the each application must be monitored and classified individually.

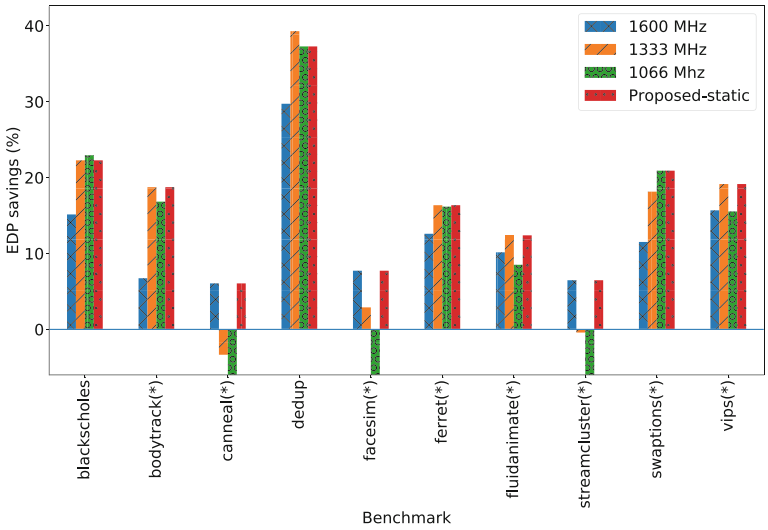


Fig. 3. EDP reduction with proposed classification using WBP compared to static-frequency baselines. Applications marked with a * have an optimal static configuration with the proposed scheme.

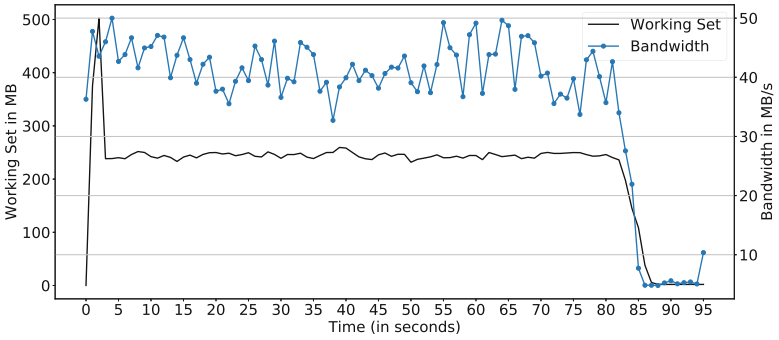


Fig. 4. Runtime memory profile of `blackscholes`. Dynamic memory access pattern calls for the exploration of a runtime policy.

7 Conclusion and Future Work

With the evolution of more heterogeneous embedded computing platforms that share a common main memory, the traditional memory bottleneck becomes even more severe in the face of balancing performance/quality and energy efficiency. In this work, we present a profiling metric, WBP (Working Set Size - Memory Bandwidth Product) that combines (1) memory bandwidth utilization and (2) working set size of running workloads. Based on the changes of EDP (energy-delay product) with frequency at different WBP, we classify applications into three categories of memory requirements. For each class, a static configuration

of the memory controller frequency is determined. Our initial results show that the static configuration correctly estimates the optimal frequency for 8 out of 10 PARSEC workloads, demonstrating the promise of our approach. The static scheme of setting memory controller frequency for each class can save EDP by 16.7% on average (37.3% maximum). Although the current work uses fixed sensing windows of 200 ms, it may not be optimal, and different window sizes require exploration. Adaptive sensing window size along with runtime policies using the proposed classification technique remains as future work.

Acknowledgement. This work was partially supported by NSF grant CCF-1704859.

References

1. Nvidia Jetson TX2 Architecture. https://devblogs.nvidia.com/jetson-tx2-delivers-twice-intelligence-edge/
2. Bienia, C., Kumar, S., Singh, J.P., Li, K.: The PARSEC benchmark suite. In: PACT, New York, New York, USA, pp. 72. ACM Press (2008)
3. David, H., Fallin, C., Gorbatov, E., Hanebutte, U.R., Mutlu, O.: Memory power management via dynamic voltage/frequency scaling. In: Proceedings of the 8th International Conference on Autonomic Computing, ICAC 2011, Karlsruhe, Germany, 14–18 June 2011, pp. 31–40 (2011)
4. Deng, Q., Meisner, D., Ramos, L., Wenisch, T.F., Bianchini, R.: Memscale: active low-power modes for main memory. SIGARCH Comput. Archit. News **39**(1), 225–238 (2011)
5. Denning, P.J.: Working sets past and present. IEEE Trans. Softw. Eng. **6**(1), 64–84 (1980)
6. Donyanavard, B., Mück, T., Sarma, S., Dutt, N.: Sparta: runtime task allocation for energy efficient heterogeneous many-cores. In: Proceedings of the Eleventh IEEE/ACM/IFIP International Conference on Hardware/Software Codesign and System Synthesis, CODES 2016, New York, NY, USA, pp. 1–10. ACM (2016)
7. Kanev, S., et al.: Profiling a warehouse-scale computer. In: Proceedings of the 42Nd Annual International Symposium on Computer Architecture, ISCA 2015, New York, NY, USA, pp. 158–169. ACM (2015)
8. David, H., Fallin, C., Gorbatov, E., Hanebutte, U.R., Mutlu, O.: Memory-aware dynamic voltage and frequency prediction for portable devices. In: The Fourteenth IEEE Internationl Conference on Embedded and Real-Time Computing Systems and Applications, RTCSA 2008, Kaohisung, Taiwan, 25–27 August 2008, pp. 229–236. Proceedings (2008)
9. Mutlu, O., Stark, J., Wilkerson, C., Patt, Y.N.: Runahead execution: an alternative to very large instruction windows for out-of-order processors. In: Proceedings of the 9th International Symposium on High-Performance Computer Architecture, HPCA 2003, Washington, DC, USA, pp. 129-140. IEEE Computer Society (2003)
10. Basireddy, K.R., Wachter, E.W., Al-Hashimi, B.M., Merrett, G.: Workload-aware runtime energy management for HPC systems. In: 2018 International Conference on High Performance Computing & Simulation, HPCS 2018, Orleans, France, 16–20 July 2018, pp. 292–299 (2018)
11. Schaller, R.R.: Moore's law: past, present and future. IEEE Spectr. **34**(6), 52–59 (1997)

12. Spiliopoulos, V., Kaxiras, S., Keramidas, G.: Green governors: a framework for continuously adaptive DVFS. In: Proceedings of the 2011 International Green Computing Conference and Workshops, IGCC 2011, Washington, DC, USA, pp. 1–8. IEEE Computer Society (2011)
13. Weiser, M., Welch, B., Demers, A., Shenker, S.: Scheduling for reduced CPU energy. In: Proceedings of the 1st USENIX Conference on Operating Systems Design and Implementation, OSDI 1994, Berkeley, CA, USA. USENIX Association (1994)
14. Wu, D., Al-Hashimi, B.M., Eles, P.: Scheduling and mapping of conditional task graph for the synthesis of low power embedded systems. IEE Proc. Comput. Digit. Tech. **150**(5), 262–273 (2003)